

Project Documentation

1. Introduction

Project Title: AI-Powered Debt Relief & Financial Recovery Platform

Team Members:

2. Project Overview

Purpose: A full-stack web application that helps borrowers who have fallen behind on loan payments understand how serious their situation is and negotiate a fair settlement with their lender.

Features:

- AI-Powered Settlement Recommendation — enter loan details (outstanding amount, EMI, overdue months, monthly income) and receive a debt-stress score and recommended settlement percentage
- Intelligent Negotiation Letter Generation — pick a letter type and tone; Google Gemini generates a personalized, lender-specific negotiation letter grounded in the borrower's real numbers
- Financial Health Tracking & Loan Analysis — a dashboard showing EMI ratio, monthly surplus, debt-stress level, and settlement history across all of a borrower's loans

3. Architecture

Frontend:

React 18 (Vite) with React Router v6 for client-side routing and custom CSS for styling (the "ink-ledger" design system). Key pages are Dashboard, AddLoan, and LoanDetail, supported by reusable components (Navbar, LoanCard, StatCard, and StressGauge — an SVG arc gauge that is the app's signature visual element).

Backend:

FastAPI (Python) with Pydantic for request/response validation and Uvicorn as the ASGI server. Routers are split by domain (loans, negotiation, dashboard), and business logic lives in two services: a pure-Python financial_analysis module (debt-stress scoring and settlement-percentage engine) and a gemini_service module (Gemini prompt construction with a template-based fallback).

Database:

SQLAlchemy ORM over SQLite. Two models: Loan (id, borrower_name, lender_name, loan_type, outstanding_amount, emi, overdue_months, monthly_income, interest_rate, created_at) and NegotiationHistory (id, loan_id FK, letter_type, generated_text, debt_stress_level, settlement_percentage, created_at).

4. Setup Instructions

Prerequisites:

- Node.js (for the React/Vite frontend)
- Python 3.10+ (for the FastAPI backend)
- A Google Gemini API key (optional — the app falls back to template-based letters if not configured)

Installation:

- Clone the repository from GitHub
- Backend: create a virtual environment, install dependencies from requirements.txt, and set GEMINI_API_KEY in a .env file (gitignored)

- Frontend: run npm install in the frontend directory
- Set the frontend's API base URL to point at the backend (local Uvicorn server or the deployed Render URL)

5. Folder Structure

Client (frontend/src):

- App.jsx — route definitions
- api.js — fetch wrapper for the backend
- index.css — design system
- components/ — Navbar, LoanCard, StatCard, StressGauge
- pages/ — Dashboard, AddLoan, LoanDetail

Server (backend):

- main.py — FastAPI app, CORS, router registration
- database.py — SQLAlchemy engine/session
- models.py — ORM models
- schemas.py — Pydantic schemas
- routers/ — loans.py, negotiation.py, dashboard.py
- services/ — financial_analysis.py, gemini_service.py

6. Running the Application

Frontend: npm run dev in the frontend directory (Vite dev server).

Backend: uvicorn main:app --reload in the backend directory.

7. API Documentation

Method	Endpoint	Description
POST	/loans	Create a loan
GET	/loans	List all loans
GET	/loans/{id}	Get one loan
GET	/loans/{id}/analysis	Loan + computed financial analysis (EMI ratio, stress score, settlement %)
DELETE	/loans/{id}	Delete a loan
POST	/negotiation/generate	Generate an AI/template negotiation letter
GET	/negotiation/history/{loan_id}	Past letters for a loan
GET	/dashboard/summary	Aggregate stats across all loans
GET	/health	Health check

8. Authentication

The current scope does not include user accounts, login, or session-based authentication — all endpoints are open. If authentication is added in a future phase, it would sit in front of the existing routers as middleware, with borrower identity linked to the Loan model.

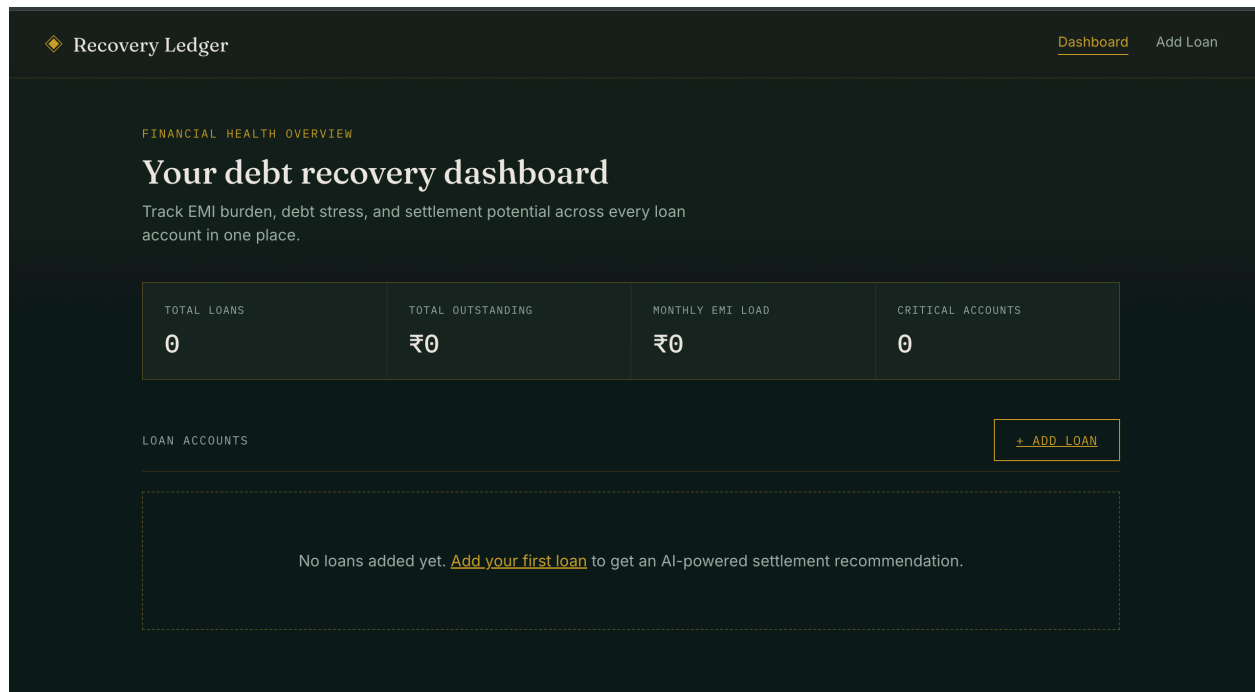
9. User Interface

Key screens: Dashboard (financial health overview), Add Loan (loan intake form), and Loan Detail (debt-stress gauge, financial analysis, and negotiation letter generation). Screenshots to be added here.

10. Testing

Functional and performance testing follows the Functional & Performance Testing Template (input validation, calculation correctness, letter generation, API response time, and load testing). User Acceptance Testing follows the UAT Template, covering loan intake, analysis, negotiation, and dashboard flows.

11. Screenshots or Demo



LOAN INTAKE

Add a loan account

Enter the outstanding balance, EMI, overdue duration, and income to generate a settlement recommendation and debt stress analysis.

BORROWER NAME	LENDER NAME
e.g. Bhavya Sri	e.g. HDFC Bank
LOAN TYPE	INTEREST RATE (% P.A.)
Personal Loan	14.5
OUTSTANDING AMOUNT (₹)	MONTHLY EMI (₹)
250000	12000
OVERDUE DURATION (MONTHS)	MONTHLY INCOME (₹)
0	35000
Generate Settlement	

[- DASHBOARD](#)

HDFC

Personal Loan · bhavya

Financial Health & Settlement Recommendation



Outstanding Amount	₹25,00,000
Monthly EMI	₹10,000
Overdue Duration	12 month(s)
Monthly Income	₹3,50,000
EMI-to-Income Ratio	2.86%
Monthly Surplus	₹3,40,000
Recommended Settlement	₹11,50,000 (46%)

Your finances are manageable but tight, with a monthly surplus of ₹3,40,000. Consider a settlement of ₹11,50,000 (46%) to resolve the loan.

AI Negotiation Letter Generator

LETTER TYPE

TONE

NEGOTIATION HISTORY

No letters generated yet for this loan.

Outstanding Amount	₹25,00,000
Monthly EMI	₹10,000
Overdue Duration	12 month(s)
Monthly Income	₹3,50,000
EMI-to-Income Ratio	2.86%
Monthly Surplus	₹3,40,000
Recommended Settlement	₹11,50,000 (46%)

Your finances are manageable but tight, with a monthly surplus of ₹340000. Consider building an emergency buffer or negotiating a lower interest rate.

GENERATE LETTER

Subject: Settlement Request for Outstanding Loan Account

Dear HDFC Team,

I am writing regarding my Personal Loan account with an outstanding balance of ₹2,500,000. Due to a change in my financial circumstances, my current EMI of ₹10,000/month now represents 2.86% of my monthly income, and the account is currently 12 month(s) overdue.

I remain committed to resolving this obligation responsibly. Based on a review of my finances, I would like to propose a one-time

NEGOTIATION HISTORY

12. Known Issues

Based on the architecture we've documented, here are the issues I'd expect to surface first:

Reliability / external dependency

- Gemini API latency or rate limits could make letter generation slow or fail under load — the template fallback should catch this, but worth confirming it triggers on timeouts, not just missing API keys
- Render's free-tier cold starts (if used) could make the first request after idle time noticeably slow

Data & concurrency

- SQLite handles concurrent writes poorly — if two requests hit `/loans` or `/negotiation/generate` at the same time, you may see "database is locked" errors under real load
- No pagination on `GET /loans` or `GET /dashboard/summary` — response time and payload size will degrade as a borrower's loan count grows

Correctness edge cases

- `monthly_income = 0` or negative EMI/outstanding values will break the EMI ratio and stress-score math (division by zero or nonsensical percentages) unless explicitly guarded in `financial_analysis.py`

- Very short overdue periods combined with very high EMI ratios could push the settlement percentage to hit its clamp (35–75%) in ways that feel arbitrary to the borrower without an explanation shown in the UI

Security / access control

- No authentication means any client can read, edit, or delete any loan by guessing/incrementing `{id}` — fine for a demo, not for real borrower data
- CORS is described as restricted to the frontend origin — worth double-checking it's not left wide open (*) from local dev testing

UX

- The StressGauge SVG may not scale cleanly on small mobile viewports
- A "Critical" stress score with no immediate next-step link (already flagged as an opportunity in the journey map) could increase borrower anxiety rather than help

13. Future Enhancements

- Borrower authentication and multi-user support
- Lender-response tracking with suggested next actions after a rejected offer
- Side-by-side comparison of multiple negotiation letter tone variants
- Migration from SQLite to a managed cloud database as usage scales

Github link: <https://github.com/bhavyainturi9/debt-relief-platform>